

Low Overhead PIM-to-PIM Communication on PCIe-based Multi-PIM Platforms for Executing Large-Scale AI Models

Mun Seong Park, Seok Young Kim, and Seon Wook Kim
School of Electrical and Computer Engineering
Korea University Seoul, Korea
{qkranstjd01, tjrdud5136, seon}@korea.ac.kr

Abstract

The emerging transformers suffer from the memory bottleneck because of their low data locality and large data sizes, and we are actively studying Processing in Memory (PIM), which computes inside memory, to overcome the bottleneck. However, as their model parameters get bigger, a single PIM device would be insufficient because of its limited memory capacity and computing resources.

In this paper, we develop a low-overhead data communication method between PIMs on a PCIe-based multi-PIM platform. We adopt an XDMA-based PIM-to-PIM (P2P) direct data communication mechanism to eliminate redundant data movement between CPU and PIMs. Our P2P achieves 1.69x, 1.70x, 1.61x, and 1.63x speedups compared to the DMA using the system memory as a buffer when transferring 16MB, 32MB, 64MB, and 128MB data, respectively.

Keywords: XDMA, Processing-in-Memory, Multi-PIM Platform, Direct Memory Access, Peer-to-Peer Communication

1. Introduction

Recently, transformers, such as BERT [1] and GPT-3 [2], have been widely used for various machine learning applications, like NLP (Natural Language Processing), video analysis, and computer vision. The inference for these models exploits low data locality since they consist of multiple layers and use parameters within one layer. Therefore, they suffer from the memory bottleneck, especially inefficiently performing on conventional CPU-based computing platforms and accelerators like GPUs having memory hierarchy.

Meanwhile, PIM has been proposed to eliminate the need to load a large amount of data to the CPU by adding a simple processing unit in/near the memory, thereby alleviating the memory bottleneck and improving performance in terms of energy and latency [3]. However, as the size of transformers grows constantly for higher accuracy, more than a single PIM device is required to infer the models. Because the bigger transformers need millions of parameters, the operand size would be too large to fit in a single PIM memory, and a single PIM also cannot exploit sufficient parallelism. Therefore, we pursue the development of a multi-PIM platform consisting of

several PIMs for larger memory capacity and higher computing parallelism.

To realize the multi-PIM platform, we use the popular PCIe interface to connect CPUs and multiple FPGA boards to emulate our baseline PIM device [4]. Two DMA engines are available in the PCIe-based multi-PIM platform, XDMA and CDMA. XDMA performs data communication between PIM devices and CPUs. On the other hand, CDMA operates for PIM execution inside one FPGA board. These separate roles for each DMA engine allow us to overlap computation and communication, i.e., hiding the communication overhead.

This paper proposes a P2P direct data communication mechanism and its driver to eliminate data movement between CPU and PIM using the system memory as a buffer. Our development shows 1.69x, 1.70x, 1.61x, and 1.63x speedups over DMA using the system memory as a buffer when transferring 16MB, 32MB, 64MB, and 128MB data, respectively.

The remainder of this paper consists of the following: Section 2 describes the bottleneck in transformer models and briefly reviews our baseline PIM architecture. In Section 3, we design and apply a DMA driver for P2P communication. In Section 4, we evaluate P2P DMA. Finally, we conclude the paper in Section 5.

2. Background

A. Data Sizes of Transformer Models

The number of parameters in large transformer models is growing exponentially every two years, while the memory capacity of a single GPU is increasing by only a factor of two. Fig.1 shows the trend in the size of the transformer models and the memory capacity [5].

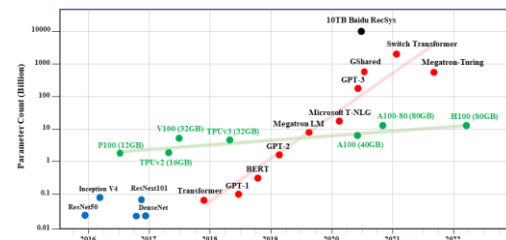


Fig. 1. The evolution of the size of transformer models with the AI accelerator memory capacity.

Transformer-based models are getting larger for higher accuracy in various tasks, and the gap with the corresponding GPU's memory capacity is widening. As a result, the memory bottleneck problem becomes more severe on conventional computing platforms.

B. Direct Memory Access, DMA

DMA is a specific-purpose engine for transferring data that allows peripheral devices to access main memory without the intervention of the CPU [6]. DMA operates through a device driver, and data movement performance varies depending on the transaction size. Additionally, CPU utilization can be increased by receiving the completion of the DMA operation through an interrupt mechanism rather than a polling mechanism. Therefore, DMA can be an efficient data movement method in multi-PIM platforms when executing large-scale models, and the data movement size is expected to be significant.

C. Processing-in-Memory, PIM

In the von Neumann architecture, the CPU executes computation by loading all data from memory and storing the result in memory. However, PIM eliminates the necessity of transferring data between memory and the processor and mitigates the memory bottleneck.

Silent-PIM [7] is our baseline PIM architecture that can use DMA as a computation offloading engine because it adopts standard memory requests as PIM commands. Silent-PIM is emulated and verified for its execution on the FPGA board. Fig. 2 shows the overall architecture, which consists of the PIM device, its memory, and the DMA engine implemented on the FPGA board.

However, CDMA [8], the DMA engine in Silent-PIM, exclusively supports the internal AXI interface within the FPGA. Consequently, utilizing CDMA, access to external host memory or other devices via PCIe is not possible. Therefore, before execution of the PIM device, the CPU should perform data copying from its memory hierarchy to the PIM device.

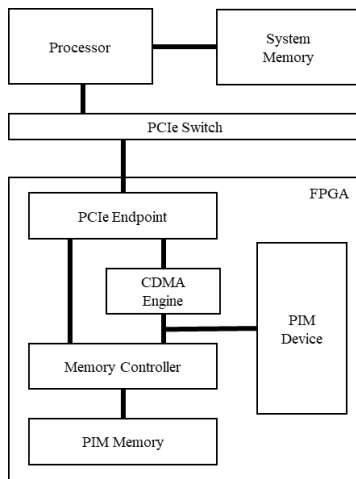


Fig. 2. Architecture of baseline PIM platform emulated on the FPGA board.

D. PCI Express, PCIe

PCIe is the standard bus interface for data communication with peripheral devices in the computer system. Due to the rapid advancements in data centers and the high demand driven by artificial intelligence, the PCIe interface is evolving rapidly. Table 1 shows the PCIe performance by generation [9]. The recently released PCIe 6.0 has increased the data

transfer speed to 64 giga transfers per second (GT/s), double that of PCIe 5.0. In our platform, PCIe 3.0 x16 is used. With DMA via PCIe bus, we can reduce the data movement overhead.

Table 1. Performance of PCI Express link.

PCIe Generation	PCIe 1.0	PCIe 2.0	PCIe 3.0	PCIe 4.0	PCIe 5.0	PCIe 6.0
Year of Release	2003	2007	2010	2017	2019	2022
Data Transfer Rate	2.5 GT/s	5.0 GT/s	8.0 GT/s	16 GT/s	32 GT/s	64 GT/s
Total Bandwidth x1 Lane	250 MB/s	500 MB/s	1 GB/s	2 GB/s	4 GB/s	8 GB/s
Total Bandwidth x2 Lanes	500 MB/s	1 GB/s	2 GB/s	4 GB/s	8 GB/s	16 GB/s
Total Bandwidth x4 Lanes	1 GB/s	2 GB/s	4 GB/s	8 GB/s	16 GB/s	32 GB/s
Total Bandwidth x8 Lanes	2 GB/s	4 GB/s	8 GB/s	16 GB/s	32 GB/s	64 GB/s
Total Bandwidth x16 Lanes	4 GB/s	8 GB/s	16 GB/s	32 GB/s	64 GB/s	128 GB/s

3. PIM-to-PIM Communication in Multi-PIM Platform

A. Overall Architecture

Fig. 3 shows the architecture in a multi-PIM platform consisting of our baseline FPGA-based PIM with an additional DMA engine on a PCIe bus. We added the XDMA IP [10] on the FPGA-based PIM for efficient data movement between the system and PIM memory, which is a configurable scatter-gather DMA implementation based on card-to-host (C2H) channels and host-to-card (H2C) channels with interrupt support. We implemented the device driver, which performs data copying from CPU to PIM using the H2C channel and from PIM to CPU using the C2H channel.

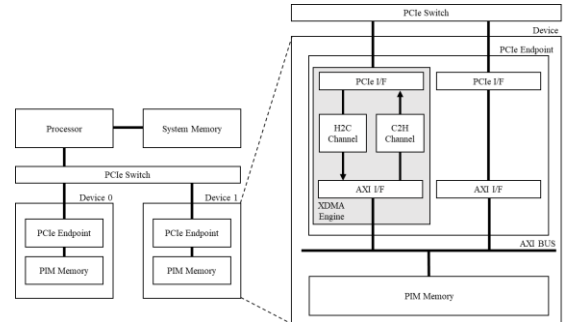


Fig. 3. Architecture of Multi-PIM platform with the XDMA engine for data communication on the PCIe bus.

B. Design of Driver for PIM-to-PIM DMA

The existing DMA driver adapts the XDMA engine to use the system memory as a buffer for data movement between PIM devices. As a result, the data movement overhead occurs because identical data are copied twice. Therefore, we developed a P2P DMA driver to eliminate the redundant data movement.

Fig. 4 illustrates the comparison of DMA mechanisms between PIM devices. Without a P2P DMA driver, data movement between PIM devices passes through the system memory, as shown in Fig. 4(a). Our driver allows a PIM device to access other PIM devices directly without exploiting the system memory as a buffer, as shown in Fig. 4(b).

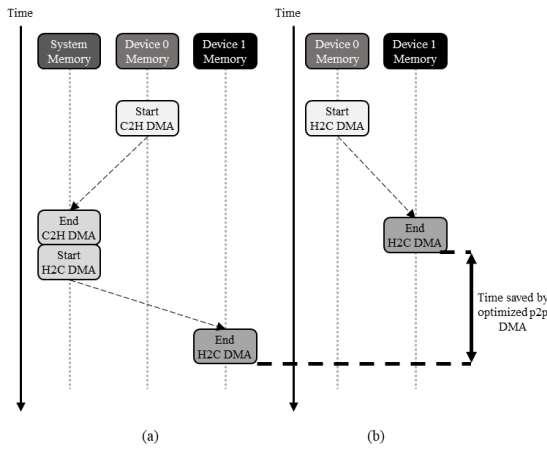


Fig. 4. Comparison of the execution flow in data movement between PIM devices via DMA.
(a) Passing through the system memory.
(b) Transferring data directly.

P2P DMA with our driver operates as follows: First, all PIM memories, recognized as IO memory by the CPU, are memory-mapped to the system memory. Second, the descriptor list is generated based on the driver's arguments for scatter-gather DMA. A descriptor contains the source address, destination address, data size, and control bits. In the case of the source address, the PCIe bus address, which can be recognized and accessed by PCIe devices that share the same PCIe switch, is used. On the other hand, the AXI bus address, which indicates the internal address of FPGA memory, is used as the destination address. Lastly, the H2C channel control register is configured to indicate the start address of the descriptor list and initiate P2P DMA. When the interrupt occurs by the DMA engine, P2P DMA terminates.

4. Evaluation

A. Methodology

Table 2 describes the configuration of the experimental environment. The data movement by the CPU is operated with the *memcpy* function [11], defined in the C standard library.

Table 2. The configuration of the experimental environment.

Experimental Environment	
Library	C standard library
OS	Ubuntu 16.04 (Linux 4.15.0-142-generic)
CPU	AMD Ryzen 7 1700 (8 cores) 3000MHz L1 I-Cache: 64KB, L1 D-Cache: 32KB L2 Cache: 512KB L3 Cache: 8MB System DRAM: 32GB
XDMA	Maximum PCIe link speed: 8.0 GT/s Transaction size per descriptor: 4KB Reference clock frequency: 100MHz AXI data width: 256-bit AXI clock frequency: 250MHz
FPGA board	Versal Prime Series VMK180 Evaluation Kit ^[12] FPGA DRAM: 1GB

B. Performance Analysis of Baseline DMA

We evaluated the performance of XDMA data movement between the system and PIM memory with 16MB dummy data. The data were stored in the source, and the data movement engine (CPU or XDMA) transferred the data to the destination. We compared the data transfer time of data movement operated by CPU and XDMA.

Fig. 5 shows the CPU and XDMA performance with the speedup XDMA. The result indicates that XDMA achieves the speedup, ranging from 3.9 to 176.8, along with the direction, compared to the data movement by CPU. The results show that DMA is an efficient communication method on the PCIe bus.

Scalability Fig. 6 shows the scalability of XDMA. The XDMA's transfer time exhibits a nearly linear increase corresponding to the data transaction size, with an average rate of approximately 1.7. The transfer time is not directly proportional to the data transaction size because DMA requires a certain amount of time for setup and completion.

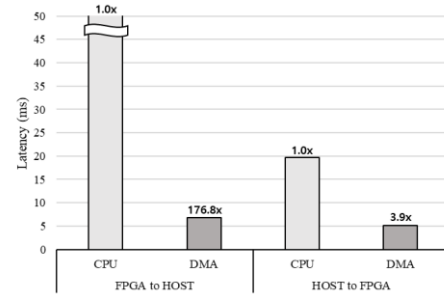


Fig. 5. Performance comparison with CPU and XDMA on 16MB data movement.

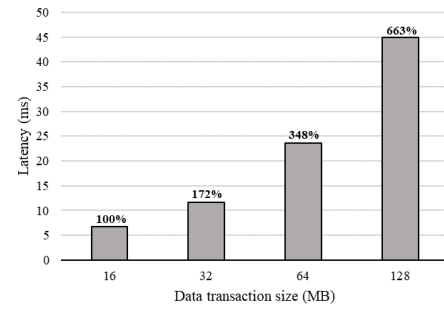


Fig. 6. Scalability of XDMA on data movement between the system and PIM memory.

C. Performance Analysis of P2P DMA

We compared the transfer time of data movement operated by the buffered DMA, which runs through the system memory with the original XDMA driver, and P2P DMA, which transfers data directly.

Fig. 7 shows their performance. The results indicate that P2P DMA resulted in a 1.69x, 1.70x, 1.61x, and 1.63x speedup compared to the buffered DMA on 16MB, 32MB, 64MB, and 128MB, respectively. The results indicate that XDMA with our modified driver performs better in P2P communication than the previous one through CPU.

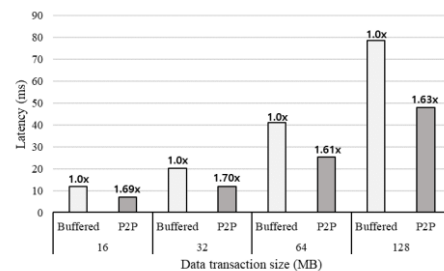


Fig. 7. Performance comparison with the buffered and P2P DMA on data movement in various transaction sizes.

5. Conclusion

We developed an efficient data communication method between FPGAs and CPUs on a multi-PIM platform, which can be utilized in large-scale AI models like LLM (Large Language Model) using XDMA. Our approach achieved 1.69x, 1.70x, 1.61x, and 1.63x speedups compared to the DMA using the system memory as a buffer when transferring 16MB, 32MB, 64MB, and 128MB data, respectively, and also showed performance scalability.

Acknowledgment

This work was partly supported by SK Hynix Inc. (Large-scale PIM Platform Research for Large Language Model, 50%) and Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. 2022-0-00441, Memory-Centric Architecture Using the Reconfigurable PIM Devices, 50%).

References

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proceedings of North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), Minneapolis, Minnesota, USA, vol. 1, pp. 4171-4186, Jun. 2019.
- [2] T. B. Brown et al., "Language models are few-shot learners," in Proceedings of Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 1877-1901. Jul. 2020.
- [3] W. J. Lee, C. H. Kim, Y. Paik, J. Park, I. Park, and S. W. Kim, "Design of Processing-'inside'-Memory Optimized for DRAM Behaviors," in IEEE Access, vol. 7, pp. 82633-82648, 2019.
- [4] W. Dong, and T. Tessil, How PCIe Tech Can Help Build Machine-Learning Accelerators, <https://www.electronicdesign.com/technologies/embedded/article/21213976/pcisig-how-pcie-tech-can-help-build-machinelearning-accelerators>, accessed:2023-10-15.
- [5] A. Gholami, Z. Yao, S. Kim, MW. Mahoney, and K. Keutzer, AI and Memory Wall, https://github.com/amirgholami/ai_and_memory_wall/tree/main, accessed:2023-10-15.
- [6] J. Corbet, A. Rubini, and G. Kroah-Hartman, Linux Device Drivers, 3rd ed, O'Reilly Media, 2005.
- [7] C. H. Kim, W. J. Lee, Y. Paik, K. Kwon, S. Y. Kim, I. Park, and S. W. Kim, "Silent-PIM: Realizing the Processing-in-Memory Computing with Standard Memory Requests," in IEEE Transactions on Parallel and Distributed Systems (TPDS), vol. 33, no. 2, pp. 251-262, Mar. 2021.
- [8] AXI Central Direct Memory Access LogiCORE IP, Product Guide, <https://docs.xilinx.com/r/en-US/pg034-axi-cdma>, accessed:2023-10-16.
- [9] PCI-SIG, PCI Express 6.0 Specification, <https://pcisig.com/pci-express-6.0-specification>, accessed:2023-10-15.
- [10] Versal Adaptive SoC DMA and Bridge Subsystem for PCI Express, Product Guide, <https://docs.xilinx.com/r/en-US/pg344-pcie-dma-versal>, accessed:2023-10-16.
- [11] The GNU C Library, Reference Manual, <https://www.gnu.org/software/libc/manual/pdf/libc.pdf>, accessed:2023-10-22.

[12] VMK180 Evaluation Board, User Guide, <https://docs.xilinx.com/r/en-US/ug1411-vmk180-eval-bd>, accessed:2023-10-20.